

SABO: Safe and Aggressive Bayesian Optimization for Automatic Legged Locomotion Controller Tuning

Anonymous Author(s)

Abstract—This paper addresses the challenge of optimizing controller parameters in complex robotic systems under safety constraints. We propose the Safe and Aggressive Bayesian Optimization (SABO) framework, which leverages Gaussian processes (GPs) to model both reward and constraint functions while using an adaptive penalty to balance the tradeoff of reward and cost. Compared to the state-of-the-art algorithm, GOSAFEOP, which relies on prior system knowledge—such as an initial safe set and the Lipschitz constant of cost functions—to construct a safe and conservative decision set, SABO synthesizes a single surrogate function by adaptively penalizing the cost functions, and encourages aggressive but safe exploration. We implement SABO in both simulation and hardware experiments on a quadruped robot. The results show that it outperforms GOSAFEOP, offering a more sample-efficient and robust solution for bridging the sim-to-real gap in dynamic environments. The code and video can be found at <https://sabo-6.github.io>.

I. INTRODUCTION

A prevalent paradigm in legged robotics research involves designing and training controllers within simulation environments prior to their deployment on physical platforms. Nonetheless, realizing robust locomotion on real hardware remains challenging due to the discrepancy between simulated and real-world conditions. Simulation environments generally assume idealized physical properties and rely on precise dynamic models, whereas real robots inevitably encounter a range of uncertainties, including sensor noise, nonlinear actuator dynamics, and environmental variability [1–3]. This discrepancy, commonly referred to as the *sim-to-real* gap, arises because the complexity of real-world dynamics cannot be fully captured or directly modeled within simulation. Consequently, controllers optimized in simulation generally require additional parameter fine-tuning on physical hardware [4].

However, manual tuning is highly inefficient in complex robotic systems [5]. As the system complexity increases (especially the number of joint actuation degrees of freedom), even low-dimensional parameterization (e.g., two-dimensional gain settings) can become challenging and time-consuming. This process also demands considerable expert knowledge, but the resulting adjustments may lack consistency even for the same system. To address these limitations, optimization-based [6] and learning-based [7] methods have been introduced as alternatives to traditional manual tuning. These automated methods offer more efficient and precise exploration of the parameter space, along with enhanced adaptability and scalability.

In general, the pursuit of optimal controller parameters is critical for ensuring stable and efficient robot locomotion

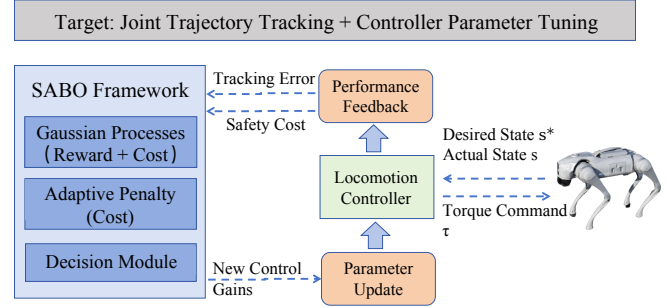


Fig. 1. Overview of the system: The locomotion controller executes joint trajectories, while tracking errors and safety-related costs are measured as feedback. These signals are provided to SABO, where Gaussian process modeling and an adaptive penalty mechanism jointly process performance and safety information. The framework then outputs updated control gains, which are iteratively applied to the controller, enabling adaptive tuning that mitigates sim-to-real gap such as friction.

in the presence of various real-world uncertainties, thereby achieving the desired control performance. If the functional relationship between controller parameters and real-world performance were explicitly known, optimal parameters could be obtained by solving an optimization problem [8]. However, since this relationship is unknown, the problem naturally falls into the category of black-box optimization. In response, Bayesian optimization has emerged as a promising data-driven methodology, offering enhanced sample efficiency over traditional grid search and gradient-based approaches [9–11]. The probabilistic framework operates under the theoretical foundation that the objective function lies in a reproducing kernel Hilbert space (RKHS) with a bounded norm, enabling the relationship between parameters and performance to be effectively modeled using Gaussian processes. Such a foundation not only enhances the efficiency of the optimization process but also provides a principled way to handle uncertainty and explore complex, high-dimensional parameter spaces. Nevertheless, a major limitation of standard Bayesian optimization is the lack of explicit consideration for safety during deployment [12]. While some parameter settings may yield high returns, they may also introduce safety risks. One straightforward approach is to impose hard constraints on the feasible parameter space. In practice, however, the constraint function is often as unknown as the reward function itself, making such an approach infeasible. To address the challenge, safe Bayesian optimization is proposed, which models the unknown constraint(s) with additional Gaussian processes and ensures safety through probabilistic confidence bounds.

A notable algorithm to solve safe Bayesian optimization

is contextual GOSAFEOP [13, 14], which is lately used for tuning joint controller parameters on the quadruped robot [15]. It provides theoretical guarantees on both safety and sample efficiency, making it well-suited for addressing the sim-to-real gap in robotic control tasks. However, a key limitation of GOSAFEOP lies in its practical implementation: it requires a known safe initial set of parameters and assumes a known Lipschitz constant for the constraint function. The method then incrementally expands the safe set via local exploration and performs global optimization by verifying safety at the boundaries. While theoretically sound, this process is complex and involves tuning multiple hyperparameters (i.e., the number of iterations for local and global phases), which contradicts the goal of automated parameter tuning in real-world applications. To address these challenges, we propose SABO, a framework for safely and smoothly updating under aggressive constraints. Our contribution is summarized as follows.

Algorithmic Design: We propose a Safe and Aggressive Bayesian Optimization (SABO) framework for the problem of automatic parameter tuning. Unlike GOSAFEOP’s *conservative* mechanism of expanding the safe set, SABO uses an *aggressive*, penalty-based framework that integrates the reward and constraint functions and adapts to be safe. This approach reduces the computational complexity associated with finding the safe set at each iteration, thereby obviating the need for prior knowledge of the Lipschitz constant, a parameter that is often impractical to ascertain. Furthermore, SABO does not require an initial safe set to guide exploration, enhancing its practicality and robustness.

Learning stability enhancement: We observe that standard Bayesian optimization often suffers from instability near convergence, where solutions oscillate among multiple feasible optima due to noise and environmental complexity. To address this, SABO incorporates a smoothness regularization term, which accelerates convergence and stabilizes the final solution, an essential property for practical deployment.

Empirical validation: We evaluate SABO with both simulation and hardware experiments, where we implement SABO on the quadruped robot *Unitree Go2* [16] and conduct a series of tests across various gait patterns. The results demonstrate that SABO outperforms GOSAFEOP in terms of safety and robustness, effectively handling dynamic environmental variations while maintaining stable locomotion. SABO achieves better sample efficiency, identifying near-optimal gait parameters in fewer iterations in experiments. Hardware experiments further confirm that controllers tuned with SABO avoid unsafe interactions and sustain stable locomotion, highlighting its effectiveness in bridging the sim-to-real gap.

II. RELATED WORKS

In the field of robotics, extensive research has been devoted to addressing the sim-to-real gap. This section provides a review of representative studies relevant to our work, highlighting their respective strengths and limitations.

Sim-to-Real Transfer Methods: Due to the discrepancies between simulation environments and the real world—arising from imperfect physical modeling, uncertain friction coefficients, and highly dynamic environmental factors—a substantial body of research focuses on addressing the *sim-to-real* gap [17]. Several representative approaches have been proposed, including dynamics randomization [18], system identification [19], and Meta RL [20]. These methods aim to enrich the simulation with sufficient variability or calibrated realism so that the policy trained in simulation remains effective when deployed on physical hardware. For instance, domain randomization [21] randomizes rendering and environmental parameters in the simulation (such as textures, lighting, camera positions, etc.), so that the model is exposed to a diverse set of scenarios during training, thereby improving its ability to generalize to real-world conditions. System identification approaches [19] focus on accurately estimating physical parameters to make the simulator more realistic, reducing the mismatch between simulated and real dynamics. Meta reinforcement learning [20] trains policies that can rapidly adapt to variations in real-world dynamics, providing greater flexibility and robustness in deployment.

Although these methods provide significant help with sim-to-real transfer, when deploying in real environments, there is still no guarantee that the robot can handle the complexities of the real world safely. Therefore, online identification remains an important step for the confirmation and refinement of the model.

Safe Bayesian Optimization: In addition to simulating real-world conditions for policy training, a more direct and practical approach is to tune controller parameters directly on physical hardware. A notable example is the application of Bayesian optimization to bipedal robots [9], where the algorithm autonomously adjusts controller parameters to maximize walking speed. The study also benchmarks several alternative search methods, demonstrating the inherent advantages of Bayesian optimization in terms of sample efficiency and convergence speed for parameter tuning.

However, standard Bayesian optimization does not account for **safety constraints**, which are critical in real-world robotic systems. To address this, SAFEOP [22] and its variants [4, 14, 15, 23] introduce the concept of a safe set: only safe parameters are evaluated. Specifically, SAFEOP assumes a safe initial policy and expands the safe set locally around this point. GOSAFEOP [14] further extends this approach by incorporating global exploration, enabling convergence to a global optimum under certain assumptions. Despite theoretical advantages, SAFEOP-type methods require numerous predefined hyperparameters as previously discussed, which limits their practicality in real-world deployments. In this work, we apply a more practical algorithm, SABO, for the problem. Through experiments, we demonstrate that our algorithm can identify optimal parameters for quadruped robots while ensuring safety.

III. PROBLEM STATEMENT

Feedback control is a fundamental concept in control systems engineering, essential for ensuring system stability, accuracy, and robustness. It operates by continuously monitoring the output s of a system, comparing it to a desired reference s^* , and using the resulting error to adjust the control input u accordingly. This dynamic feedback process enables the system to automatically correct deviations caused by internal uncertainties (i.e., model discrepancy) or external disturbances (i.e., noise), making it especially critical for model-based controllers in mitigating the *sim-to-real* gap. Specifically, the feedback system can be formulated as:

$$u = \pi_x(u^*, s^*, s). \quad (1)$$

where u^* denotes the ideal control input (i.e., joint torque), and the feedback policy π is parameterized by x within the domain $\mathcal{X}$. The strategy allows the system to adjust its inputs in real-time, correcting errors and stabilizing performance. In legged locomotion, a widely used feedback-based strategy is PD control, where the controller parameter x is represented by k_p and k_d in this context. The formulation of PD control law is given by:

$$u = u^* + k_p(\bar{s}^* - \bar{s}) + k_d(\dot{\bar{s}}^* - \dot{\bar{s}}), \quad (2)$$

where $k_p \in \mathbb{R}^{d_u \times d_{\bar{s}}}$ and $k_d \in \mathbb{R}^{d_u \times d_{\dot{\bar{s}}}}$. We use $\bar{s}$ to represent the joint-level states. In this context, the term $\bar{s}^* - \bar{s}$ denotes the joint position error and $\dot{\bar{s}}^* - \dot{\bar{s}}$ represents the joint velocity error. A key practical challenge in applying this control law lies in selecting appropriate values for the gain matrices k_p and k_d , which is often non-trivial and typically relies on heuristic tuning or expert intuition. The manual process can be time-consuming, suboptimal, and especially unsafe in real-world systems where inappropriate gain values may lead to instability or unsafe behavior. To overcome these limitations, we reformulated this process as a constrained optimization problem, wherein the gains are treated as decision variables that maximize a task-specific function, while subject to a safety constraint:

$$\max_{x \in \mathcal{X}} f(x) \quad \text{s.t.} \quad g(x) \leq v. \quad (3)$$

In this formulation, x encodes the control gains, $f(x) : \mathcal{X} \rightarrow \mathbb{R}$ denotes a trajectory tracking function, and $g(x) : \mathcal{X} \rightarrow \mathbb{R}$ captures the safety-critical constraint that must be satisfied during system operation. Thus, our goal is to identify appropriate controller parameters such that the system can track desired trajectories in real time while mitigating additional errors introduced by the complexity of real-world dynamics. However, both functions in (3) are black-box functions: their analytical forms are unknown and can only be accessed through noisy observations of the system's actual responses. Consequently, solving this constrained optimization problem inevitably requires repeated interactions with the robot, rendering the optimization process inherently data-driven.

This data-driven nature introduces additional practical constraints. First, since each function evaluation corresponds

to a real or simulated rollout, **sample efficiency** becomes crucial—especially when experiments are performed on physical systems where interaction is costly or limited. Second, and more critically, **safety during exploration** must be carefully considered, as unsafe gain parameters may lead to irreversible hardware damage. These considerations motivate the use of methods such as Safe Bayesian optimization, which balance performance improvement with risk-aware exploration in a data-efficient manner.

IV. SABO ALGORITHM

In this section, we introduce SABO (Safe and Aggressive Bayesian Optimization), our proposed algorithm for safe parameter tuning. The method combines Gaussian Process Regression (GPR) to model the unknown objective and constraint functions with an adaptive, penalty-based framework that efficiently solves the constrained optimization problem (3) in an online setting. We further demonstrate the advantages of SABO compared with the previous state-of-the-art (SOTA) methods in terms of sample efficiency, robustness, and ease of deployment. In the following, we describe these components in detail.

A. Gaussian Process Modeling

Gaussian Process [24] is a non-parametric Bayesian approach widely used for modeling unknown or expensive-to-evaluate black-box functions. For the target function, Gaussian Process Regression (GPR) iteratively updates its posterior distribution as new observations are acquired. This update refines the model's confidence intervals and allows the optimization algorithm to make informed decisions: exploring uncertain regions to reduce model uncertainty, or exploiting regions with high predicted performance. Over time, this process guides the search towards the global optimum [25], where the online learning capability aligns well with our goal of adaptively tuning controller parameters in real-time environments.

In our SABO framework, Gaussian Process regression is employed to jointly approximate the objective and constraint functions based on observed data. Specifically, let $\mathbf{h} = [f, g]$ denote the vector-valued function combining both performance and safety objectives. We model $\mathbf{h}(x)$ over the decision space $\mathcal{X}$ as a multivariate Gaussian process:

$$\mathbf{h}(x) \sim \mathcal{GP}(\mu^{\mathbf{h}}(x), k^{\mathbf{h}}(x, x')),$$

where $\mu^{\mathbf{h}}(x) = \mathbb{E}[\mathbf{h}(x)]$ denotes the prior mean function and $k^{\mathbf{h}}(x, x') = \text{cov}(\mathbf{h}(x), \mathbf{h}(x'))$ is the covariance function, encoding prior assumptions about the smoothness and structure of the function space.

At $t-1$ iteration, we collect a interaction history $\mathcal{D}_{1:t-1} = (\mathbf{x}_i, \mathbf{h}_i)_{i=1}^{t-1}$, where $\mathbf{x}_i$ denotes the decision taken at iteration i , and $\mathbf{h}_i = [f_i, g_i]$ represents the corresponding observed function values. Given this data, the Gaussian process pos-

terior at a new point $\mathbf{x}$ can be computed in closed form:

$$\begin{aligned}\mu_t^h(\mathbf{x}) &= k_t^h(\mathbf{x})^\top (V_t^h(\lambda))^{-1} h_{1:t}, \\ k_t^h(\mathbf{x}, \mathbf{x}') &= k^h(\mathbf{x}, \mathbf{x}') - k_t^h(\mathbf{x})^\top (V_t^h(\lambda))^{-1} k_t^h(\mathbf{x}'), \\ \sigma_t^h(\mathbf{x}) &= \sqrt{k_t^h(\mathbf{x}, \mathbf{x})},\end{aligned}$$

where $[K_t^h]_{(i,j)} = k^h(\mathbf{x}_i, \mathbf{x}_j)$, $i, j \in \{1, \dots, t-1\}$, $V_t^h(\lambda) = K_t^h + \lambda I$, $\lambda = 1 + 3/T$, $h_{1:t} = [h_1, \dots, h_t]^\top$, and $k_t^h(\mathbf{x}) = [k^h(\mathbf{x}_1, \mathbf{x}), \dots, k^h(\mathbf{x}_{t-1}, \mathbf{x})]^\top$.

These posterior quantities not only provide estimates for the objective and constraint functions at any candidate decision $\mathbf{x}$, but also quantify the uncertainty of the predictions through the variances. Nevertheless, even with this probabilistic characterization of the decision space, the central challenge remains: **how should safe and smooth decisions be made?** In particular, the algorithm must determine (i) when to explore uncertain regions, (ii) when to exploit high-reward regions, and (iii) when to prioritize constraint satisfaction. Balancing these competing goals in a unified manner is non-trivial. To address this, we propose the SABO algorithm, a safe Bayesian optimization approach that adaptively manages this trade-off through a unified surrogate objective.

B. SABO Algorithm

In the algorithm, SABO first employs GP-UCB/LCB to estimate the reward function optimistically and the cost function pessimistically, which is defined as:

$$\begin{aligned}\hat{f}_t(\mathbf{x}) &= \mu_t^f(\mathbf{x}) + \beta_t^f \sigma_t^f(\mathbf{x}), \\ \check{g}_t(\mathbf{x}) &= \mu_t^g(\mathbf{x}) - \beta_t^g \sigma_t^g(\mathbf{x}).\end{aligned}\quad (4)$$

where the quantities β_t^f and β_t^g are instance-dependent Bayesian constructs, which depend on the kernel function $k(\mathbf{x})$ and decision domain $\mathcal{X}$. Their values will be specified in Section V. Intuitively, in GP-UCB/LCB, $\hat{f}_t(\mathbf{x})$ and $\check{g}_t(\mathbf{x})$ serve as the upper and lower confidence bounds for the real reward and cost function, respectively. Based on the estimations, SABO makes the decision by maximizing a surrogate function:

$$\mathbf{x}_t = \arg \max_{\mathbf{x} \in \mathcal{X}} \hat{f}_t(\mathbf{x}) - Q_t \check{g}_t^+(\mathbf{x}) - \alpha \|\mathbf{x} - \mathbf{x}_{t-1}\|_2^2 \quad (6)$$

where $\check{g}_t^+(\mathbf{x}) = \max(0, \check{g}_t(\mathbf{x}))$ represents the predicted violation of the constraint (only penalizing unsafe actions), Q_t is an adaptive penalty parameter to control constraint violation. The final term introduces temporal smoothness by penalizing abrupt changes in the decision variable, with $\alpha \geq 0$ controlling the regularization strength. Incorporating the temporal regularization term α ensures that the final solution converges smoothly, mitigating oscillations between multiple candidate parameters. This is particularly important in the final stages of the algorithm, where several candidate points may have similar estimated optimality values, and the presence of observation noise can cause repeated switching among them over many iterations, which is a behavior that is detrimental in practice and commonly observed in standard

Bayesian optimization methods. A detailed demonstration is provided in table I.

The optimization in SABO simultaneously balances three objectives: 1) maximize the estimated reward via $\hat{f}_t(\mathbf{x})$; 2) minimize the (penalized) constraint violation through $Q_t \check{g}_t^+(\mathbf{x})$; and 3) stabilize decision updates using the temporal regularization term $\alpha \|\mathbf{x} - \mathbf{x}_{t-1}\|_2^2$. After submitting $\mathbf{x}_t$ and receiving feedback from the environment, SABO updates the penalty parameter Q_t to reflect the violation level:

$$Q_{t+1} = \max(Q_t + c_t^+(\mathbf{x}_t), \eta_t). \quad (7)$$

where $c_t^+(\mathbf{x}_t)$ denotes the instantaneous constraint violation at iteration t , and $\eta_t = \sqrt{t}$ serves as a lower bound to ensure that the penalty does not vanish over time. This dynamic update of Q_t is central to SABO's double-adaptive mechanism. When constraint violations are small or absent, Q_t remains low, allowing the algorithm to prioritize reward maximization and exploration. Conversely, when violations accumulate, Q_t increases, steering the algorithm toward safer decisions to enforce constraint satisfaction. Combined with the temporal regularization term α , this mechanism ensures smooth, stable adaptation while efficiently balancing the trade-off between reward and safety. We summarize the complete algorithm as follows.

SABO Algorithm

Initialization: $Q_1 = 1$ and $\eta_t = \sqrt{t}$. Gaussian process model (Θ_1^f, Θ_1^g) .

For $t = 1, \dots, T$,

- **Estimate reward and cost functions:**

$$\hat{f}_t(\mathbf{x}), \check{g}_t(\mathbf{x}) = \text{UCB/LCB}(\Theta_t^f, \Theta_t^g)$$

- **Decide next parameter:** choose $\mathbf{x}_t$ based on (6).
- **Receive feedback:** reward $r_t(\mathbf{x}_t)$ and cost $c_t(\mathbf{x}_t)$.
- **Update the penalty:** calculate Q_{t+1} based on (7).
- **Update GP model:**

$$\Theta_{t+1}^f, \Theta_{t+1}^g = \text{UCB/LCB}(\Theta_t^f, \Theta_t^g, \{\mathbf{x}_t, r_t, c_t\}). \quad (8)$$

Intuitively, SABO maintains and updates surrogate models of both the reward and constraint functions using past data. The core idea of SABO lies in its **double-adaptive mechanism**: i) decisions $\mathbf{x}_t$ are selected adaptively using current posterior estimates and penalties. ii) the penalty parameter Q_t evolves adaptively in response to empirical constraint violations. The joint adaptation enables SABO to efficiently solve the constrained optimization problem in an **online** and **safety-aware** manner.

C. Comparison with State-of-the-Art Method

The most prominent state-of-the-art (SOTA) method for safe Bayesian optimization is GOSAFEOP. It identifies the global optimum by alternating between local and global exploration, under the assumptions of a known safe initial set and a known Lipschitz constant for the constraint function.

As discussed previously, while this method is theoretically elegant and provides formal safety and convergence guarantees, we argue that its practical applicability in dynamic real-world environments is limited. First, the assumption of a known safe initial set and an accurate Lipschitz constant may not hold in practice, especially in complex systems where such prior knowledge is unavailable or difficult to obtain. Second, the bi-level optimization structure (local vs. global exploration) introduces multiple hyperparameters that must be manually tuned. For instance, the number of local iterations before a global expansion step must be predefined (denoted as n). An excessively large or small value of n can negatively impact the algorithm's exploration and convergence. Therefore, GOSAFEPT requires careful manual tuning of multiple hyperparameters, which contradicts the goal of achieving fully autonomous and adaptive parameter optimization.

Compared with SAFEPT-type methods, SABO offers a simpler and more practical solution. Rather than relying on hand-crafted exploration phases or strong prior knowledge of safe sets and Lipschitz constants, SABO automatically balances reward maximization and constraint satisfaction through the adaptive penalty Q_t . The framework introduces only a single additional hyperparameter, α , which regularizes the smoothness of decision updates and mitigates abrupt parameter changes that could destabilize the system. As demonstrated in Section V, SABO achieves strong performance even with $\alpha = 0$, while larger values of α improve convergence stability and robustness in dynamic environments.

V. EXPERIMENTS

In this section, we present the experimental results evaluating the performance of SABO and GOSAFEPT using *Unitree Go2* [16] in both simulation and hardware. We follow the control framework in [26], which combines Nonlinear Predictive Control (NMPC) and Whole Body Controller (WBC). NMPC uses discrete-time control barrier functions (DCBF) constraints for better robot locomotion, while WBC ensures stable low-level torque tracking during locomotion. The update frequencies for NMPC and WBC are 30 Hz and 500 Hz, respectively. And we use the following objective and constraint functions:

$$f(\mathbf{x}) = - \sum_{t>0} \|\bar{\mathbf{s}}^*(t) - \bar{\mathbf{s}}(t, \mathbf{x})\|_2^2, \quad (9)$$

$$g(\mathbf{x}) = \max_{t>0} \|\bar{\mathbf{s}}^*(t) - \bar{\mathbf{s}}(t, \mathbf{x})\|_2^2 - v, \quad (10)$$

where t is collecting time horizon and v is the cost threshold. $f(\mathbf{x})$ denotes the tracking error between the desired joint-level state $\bar{\mathbf{s}}^*$ and actual state $\bar{\mathbf{s}}$ during the collection time. The constraint function $g(\mathbf{x})$ restricts the instantaneous error from surpassing the threshold throughout the entire duration.

A. Experimental Setup

In the experiments, we set the collecting time horizon to 2 seconds, i.e., for each candidate parameter $\mathbf{x}$, we collect

both the tracking error and the constraint violation within a $t = 2s$ window to evaluate performance. The cost threshold is fixed at $v = 0.3$. In this control framework, the decision variable $\mathbf{x}$ corresponds to the parameters of a proportional-derivative (PD) controller, denoted as $[\mathbf{k}_p, \mathbf{k}_d]$. Since the Unitree Go2 robot has four legs and each leg consists of three actuated joints (Hip, Thigh, and Calf), both $\mathbf{k}_p$ and $\mathbf{k}_d$ are 12-dimensional vectors.

For computational tractability of Bayesian optimization, however, we adopt a simplified setting where all joints share the same controller parameter. While this restriction may result in a slight performance loss compared to a fully joint-specific tuning, our experiments show that this simplification is sufficient to obtain satisfactory solutions, and it is consistent with most practical controller-tuning approaches reported in the literature. Importantly, our algorithm is also flexible to be combined with dimensionality-reduction techniques in Bayesian optimization if needed [27–29].

Based on prior knowledge of controller design, we restrict the search space to $\mathcal{A} = [0, 9]^2$, as parameter values greater than 10 typically result in instability or invalid behavior. For Bayesian hyperparameters, we use the Matern kernel function [13] and set $\beta_t^f = \beta_t^g = 16$. The kernel lengthscale is set to 0.1 for simulator experiments and 0.3 for hardware experiments. The optimization goal is to identify optimal PD parameters for multiple gaits using as few evaluations as possible out of a large candidate space (potentially tens of thousands of samples).

Regarding GOSAFEPT's hyperparameters (e.g., the number of local and global exploration steps, as well as the Lipschitz constant), we adopt the values provided in the official open-source implementation [15] and make minor adjustments tailored to our setup. For instance, the number of iterations for the local and global exploration phases is fixed to $n_l = 10$ and $n_g = 5$, respectively, ensuring sufficient local exploitation while allowing for global boundary expansion. Finally, we conduct experiments across various gaits, including trot, flying trot, and pace, to comprehensively evaluate the performance and robustness of the proposed algorithm.

Through the experiments, we aim to address three key questions: (i) Can SABO-type algorithms achieve high performance without requiring prior knowledge of safe sets or Lipschitz constants? (ii) Does the introduction of the regularization term improve the stability and robustness of SABO by facilitating faster and more reliable convergence? (iii) Can SABO deliver robust performance when deployed on real hardware under complex, dynamic conditions?

B. Simulation Results

In the simulation experiments, we compare the performance of GOSAFEPT and SABO with different regularization terms. For SABO-type algorithms, each trial is initialized from a randomly selected point in the search space. In contrast, for GOSAFEPT, we follow its algorithmic requirements by starting from a manually tuned parameter that is known to be safe. The corresponding learning curves are shown in Figure 2. Each curve represents the mean

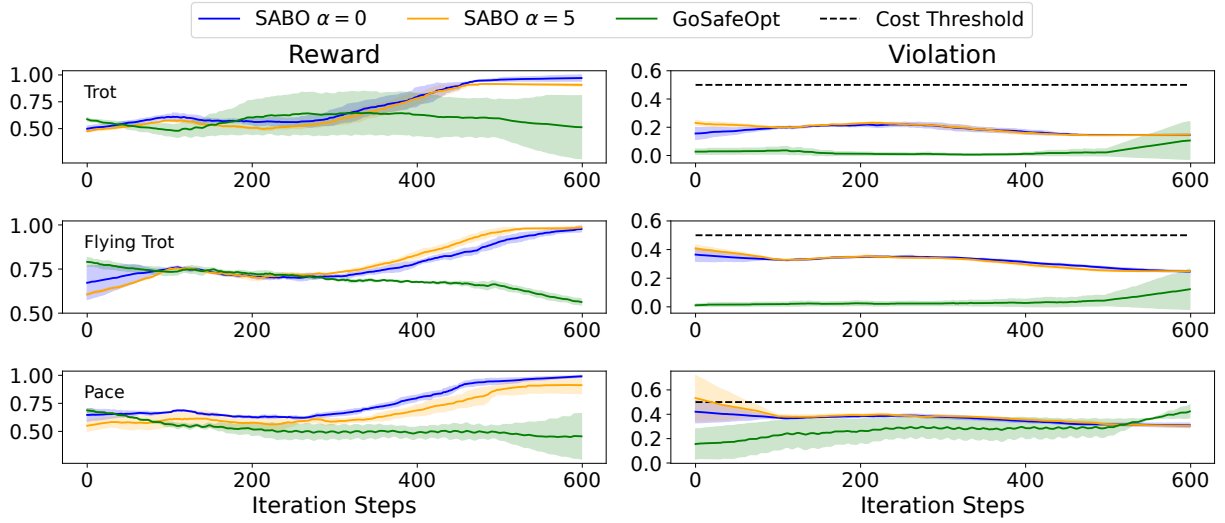


Fig. 2. Comparison of learning curves for GOSAFEPT and SABO with different regularization terms across multiple gaits. Each curve shows the mean performance over five random experiments, with shaded areas denoting one standard deviation. For SABO, trials are initialized from randomly sampled points, whereas GOSAFEPT starts from a manually selected safe parameter. Performance is normalized relative to the best final reward. SABO ($\alpha = 0$ and $\alpha = 5$) consistently achieves high rewards while suppressing safety violations, even when initialized from unsafe configurations. In contrast, GOSAFEPT remains overly conservative, restricted to the provably safe set.



Fig. 3. Sensitivity analysis of the regularization parameter α . Bars indicate the mean final reward over five random seeds, and error bars represent standard deviation. Results demonstrate that larger α values have minimal impact on asymptotic performance but contribute to faster and more stable convergence.

performance across five random seeds, with shaded regions indicating one standard deviation. For fair comparison, the best performance achieved across all algorithms is normalized to a reward value of 1, and the performance of other methods is evaluated relative to this baseline.

TABLE I
INSTABILITY RATE IN THE LAST 100 ITERATION STEPS

	Trot	Flying Trot	Pace
SABO $\alpha = 0$	0.273	0.487	0.300
SABO $\alpha = 5$	0.000	0.020	0.140
GOSAFEPT	0.645	0.682	0.784

We observe that, across all gaits, the learning dynamics follow the "exploitation-exploration-exploitation" pattern: the reward first rises, then temporarily drops during exploration, and eventually converges toward the optimum. Notably, both SABO $\alpha = 0$ and SABO $\alpha = 5$ consistently reach high performance while keeping violations strictly below the threshold. Even when initialized from an unsafe configuration (e.g., pace gait), SABO rapidly suppresses violations and converges within 600 iterations. In contrast, GOSAFEPT

remains overly conservative, restricted to the provably safe set, and fails to complete global exploration within 600 iterations.

We also observe that, while SABO with $\alpha = 0$ achieves slightly high rewards, it exhibits unstable convergence, with solutions oscillating between multiple parameter values. Such behavior is undesirable in practice, as we prefer the algorithm to settle on a single stable solution. To compare the convergence performance of different methods, we examined the parameter trajectories during the final 100 iterations. As shown in Table I, SABO with $\alpha = 5$ significantly reduces the rate of parameter switching compared with $\alpha = 0$, demonstrating that the regularization term accelerates convergence and stabilizes the solution—an important property for real-world deployment. By contrast, GOSAFEPT shows no clear signs of convergence, as it remains in the exploration phase.

Finally, we conducted a sensitivity analysis of the regularization term. Figure 3 reports the final rewards achieved at the last iteration under different values of α . The results show that increasing α has only a limited effect on the asymptotic optimality of the algorithm, and in the case of the flying trot gait, the impact is almost negligible. However, as demonstrated earlier, larger α values significantly improve

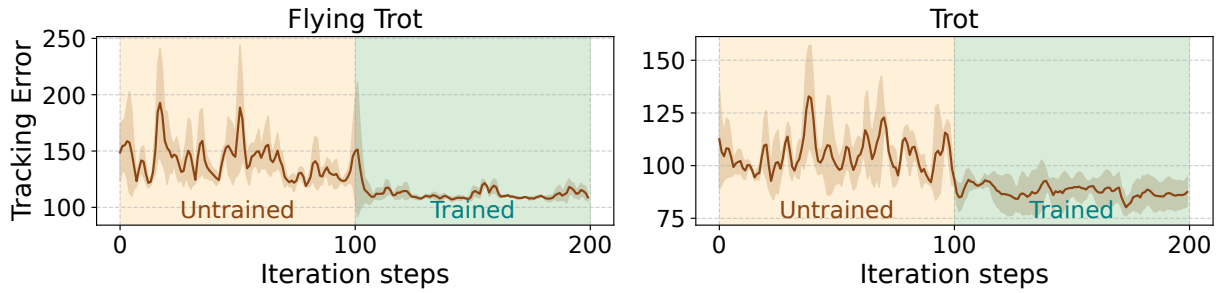


Fig. 4. Hardware evaluation of SABO across different gaits. The yellow region corresponds to manually tuned PD controller parameters, and the green region to parameters optimized by SABO. Each bar shows the mean joint trajectory tracking error over multiple runs, with error bars indicating one standard deviation. SABO consistently reduces tracking error and converges within a few dozen iterations.

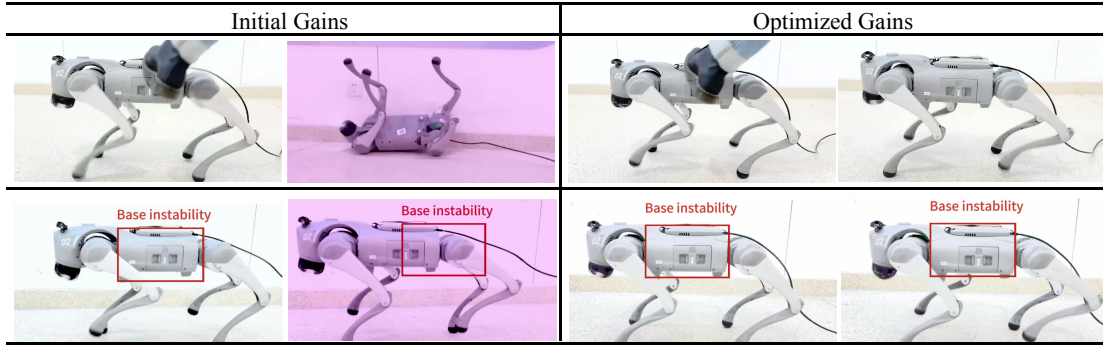


Fig. 5. Qualitative comparison of robot behavior before (left) and after (right) SABO optimization. The SABO-optimized controller improves robustness against external pushes and enhances base stability, demonstrating the framework’s effectiveness in real-world deployment.

convergence speed and stability. This confirms that the role of α is primarily to enhance robustness and accelerate convergence, without compromising the ultimate performance of the algorithm.

C. Hardware Results

We further evaluated SABO in hardware experiments. Figure 4 presents its performance across different gaits. The yellow region corresponds to manually tuned PD controller parameters, while the green region shows the performance of parameters optimized by SABO. The results indicate that the optimized parameters substantially reduce the joint trajectory tracking error compared with the baseline. Notably, SABO achieves convergence within only a few dozen iterations, highlighting its strong sample efficiency.

To better highlight practical improvements, we also recorded the robot’s behavior before and after optimization. In Figure 5, the SABO-optimized controller exhibits markedly enhanced robustness under external pushes and improved base stability, validating both the effectiveness and practicality of our framework in real-world settings.

VI. CONCLUSION

We present the Safe and Aggressive Bayesian Optimization (SABO) algorithm to the problem of tuning controller parameters in robotic systems under safety constraints. SABO models both reward and constraint functions using Gaussian processes, integrating them through an adaptive

penalty to ensure that safety constraints are consistently satisfied. The adaptive penalty-based framework enhances SABO’s sample efficiency and practical applicability compared to the state-of-the-art algorithm. Specifically, compared with GOSAFEOP, SABO requires much less prior knowledge, as it does not rely on an accurately specified initial safe set or Lipschitz constant—both of which are typically difficult to obtain in real-world environments. We implement SABO in both simulations and hardware experiments, and the results demonstrate that SABO outperforms GOSAFEOP across a range of scenarios, offering a more sample-efficient and robust solution in bridging the sim-to-real gap in safety-critical robotics.

REFERENCES

- [1] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, “Learning agile and dynamic motor skills for legged robots,” *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.
- [2] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, “Sim-to-real: Learning agile locomotion for quadruped robots,” *arXiv preprint arXiv:1804.10332*, 2018.
- [3] J. Rothfuss, B. Sukhija, L. Treven, F. Dörfler, S. Coros, and A. Krause, “Bridging the sim-to-real gap with bayesian inference,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 10 784–10 791.

- [4] F. Berkenkamp, A. Krause, and A. P. Schoellig, “Bayesian optimization with safety constraints: safe and automatic parameter tuning in robotics,” *Machine Learning*, vol. 112, no. 10, pp. 3713–3747, 2023.
- [5] A. Schperberg, S. Di Cairano, and M. Menner, “Auto-tuning of controller and online trajectory planner for legged robots,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7802–7809, 2022.
- [6] B. Siciliano, L. Sciacivco, L. Villani, and G. Oriolo, *Robotics: modelling, planning and control*. Springer, 2009.
- [7] C. K. Williams and C. E. Rasmussen, *Gaussian processes for machine learning*. MIT press Cambridge, MA, 2006, vol. 2, no. 3.
- [8] R. Sadeghian and M. T. Masouleh, “Controller tuning based on optimization algorithms of a novel spherical rolling robot,” *Journal of Mechanical Science and Technology*, vol. 30, no. 11, pp. 5207–5216, 2016.
- [9] R. Calandra, A. Seyfarth, J. Peters, and M. P. Deisenroth, “Bayesian optimization for learning gaits under uncertainty: An experimental comparison on a dynamic bipedal walker,” *Annals of Mathematics and Artificial Intelligence*, vol. 76, pp. 5–23, 2016.
- [10] Y. Sui, V. Zhuang, J. Burdick, and Y. Yue, “Stagewise safe bayesian optimization with gaussian processes,” in *International conference on machine learning*. PMLR, 2018, pp. 4781–4789.
- [11] S. Vakili, K. Khezeli, and V. Picheny, “On information gain and regret bounds in gaussian process bandits,” in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2021, pp. 82–90.
- [12] C. König, M. Turchetta, J. Lygeros, A. Rupenyan, and A. Krause, “Safe and efficient model-free adaptive control via bayesian optimization,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 9782–9788.
- [13] A. Krause and C. Ong, “Contextual gaussian process bandit optimization,” *Advances in neural information processing systems*, vol. 24, 2011.
- [14] B. Sukhija, M. Turchetta, D. Lindner, A. Krause, S. Trimpe, and D. Baumann, “Gosafeopt: Scalable safe exploration for global optimization of dynamical systems,” *Artificial Intelligence*, vol. 320, p. 103922, 2023.
- [15] D. Widmer, D. Kang, B. Sukhija, J. Hübotter, A. Krause, and S. Coros, “Tuning legged locomotion controllers via safe bayesian optimization,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2444–2464.
- [16] Unitree Robotics, <https://www.unitree.com/en/go2>.
- [17] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-real transfer in deep reinforcement learning for robotics: a survey,” in *2020 IEEE symposium series on computational intelligence (SSCI)*. IEEE, 2020, pp. 737–744.
- [18] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 3803–3810.
- [19] K. Kristinsson and G. A. Dumont, “System identification and control using genetic algorithms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 22, no. 5, pp. 1033–1046, 2002.
- [20] K. Arndt, M. Hazara, A. Ghadirzadeh, and V. Kyriki, “Meta reinforcement learning for sim-to-real domain adaptation,” in *2020 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2020, pp. 2725–2731.
- [21] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 23–30.
- [22] Y. Sui, A. Gotovos, J. Burdick, and A. Krause, “Safe exploration for optimization with gaussian processes,” in *International conference on machine learning*. PMLR, 2015, pp. 997–1005.
- [23] D. Baumann, A. Marco, M. Turchetta, and S. Trimpe, “Gosafe: Globally optimal safe robot learning,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 4452–4458.
- [24] C. K. Williams and C. E. Rasmussen, *Gaussian processes for machine learning*. MIT press Cambridge, MA, 2006, vol. 2.
- [25] C. Fiedler, C. W. Scherer, and S. Trimpe, “Practical and rigorous uncertainty bounds for gaussian process regression,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 8, 2021, pp. 7439–7447.
- [26] Q. Liao, Z. Li, A. Thirugnanam, J. Zeng, and K. Sreenath, “Walking in narrow spaces: Safety-critical locomotion control for quadrupedal robots with duality-based optimization,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 2723–2730.
- [27] K. Kandasamy, J. Schneider, and B. Póczos, “High dimensional bayesian optimisation and bandits via additive models,” in *International conference on machine learning*. PMLR, 2015, pp. 295–304.
- [28] C.-L. Li, K. Kandasamy, B. Póczos, and J. Schneider, “High dimensional bayesian optimization via restricted projection pursuit models,” in *Artificial Intelligence and Statistics*. PMLR, 2016, pp. 884–892.
- [29] D. Ulmasov, C. Baroukh, B. Chachuat, M. P. Deisenroth, and R. Misener, “Bayesian optimization with dimension scheduling: Application to biological systems,” in *Computer Aided Chemical Engineering*. Elsevier, 2016, vol. 38, pp. 1051–1056.